What is Testing

Software Testing is the process of evaluating a software application to find defects, verify requirements, and ensure it works as expected before release. It involves executing the system (manually or via automation) and checking whether actual behavior matches expected behavior.

You can think of it as: *"checking software quality by validating and breaking it before users do."*

Why software testing is necessary

- **Finds defects early** → reduces cost of fixing bugs after release
- **Ensures requirements are met** → builds the right product, not just a working one
- **Improves product quality** → stability, performance, usability
- **Prevents business loss** → avoids failures in production (money, reputation, users)
- **Enhances security** → detects vulnerabilities before attackers do
- **Builds user trust** → reliable software increases adoption

In short: without testing, software is a "guess"; with testing, it becomes a "reliable product."

Quality and Quality Assurance

- **Quality**: A degree of excellence of a product or a service.
- **Quality assurance (QA)**: is any systematic process of determining whether a product or service meets specified requirements.

Testing Objectives

1. Functional Correctness

Checks whether the software works according to requirements and business rules.

- Valid inputs produce correct outputs
- Logic and calculations are accurate
- Handles edge cases properly

2. Authorization

Ensures users can only access permitted features and data.

- Role-based access control works correctly
- Protected APIs cannot be accessed without permission
- Unauthorized actions are blocked

3. Service Level (Performance & Reliability)

Measures how well the system performs under real usage.

- Fast response time
- Handles expected load
- Stable under stress
- High availability

4. Usability

Evaluates how easy the system is to use for real users.

- Simple and clear UI
- Smooth navigation
- Clear error messages
- User-friendly workflow

Software Testing Misconceptions

1. Without knowing programming, anyone can do software testing

Fact:

Basic UI manual testing can be done using common sense and domain understanding, but real testing goes far beyond that.

Whitebox testing, API testing, client-server validation, data testing, and test automation require strong programming and database knowledge to ensure correct verification of system behavior.

2. QA is getting replaced by AI

Fact:

AI is reducing repetitive manual testing tasks, but testing itself is not disappearing.

Manual QA roles are shrinking, while demand for AI-assisted SDET and technical QA engineers is increasing. The focus is shifting toward engineers who can design, automate, and validate quality using AI tools.

3. Testing is only about finding bugs

Fact:

Bug detection is an important objective, but not the only one.

The main goal of testing is ensuring overall software quality. This requires technical understanding to validate functional and non-functional aspects such as performance, security, usability, and reliability of production-grade systems.

4. Automation testing tools are enough, programming is not required

Fact:

Automation tools only provide the framework; real value comes from coding ability.

Test scripts, framework design, assertions, debugging failures, and maintaining test suites all require solid programming skills

QA Generic Skills

The following skills are particularly relevant for testers:

1. Testing Knowledge

Understanding testing concepts, test levels, test types, and test techniques to test software effectively.

2. Attention to Detail

Being careful, thorough, curious, and methodical to find defects that others may miss.

3. Communication Skills

Explaining issues clearly, listening actively, working with the team, and reporting defects in an understandable way.

4. Analytical & Critical Thinking

Analyzing requirements, questioning unclear logic, identifying risks, and thinking creatively to find hidden problems.

5. Technical Knowledge

Using tools, APIs, databases, logs, automation, and other technical skills to test more efficiently.

6. Domain Knowledge

Understanding the business domain so testers can validate whether the software solves real user needs correctly.

Simple Summary

A good QA is not just a bug finder. A good QA needs **testing knowledge, sharp observation, communication, analytical thinking, technical skill, and business understanding**.

Error, Defect, Bug and Failure

Error:

Human beings make errors (mistakes), or misunderstanding requirements, which produce defects (faults). Humans make errors for various reasons, such as time pressure, complexity of work products, processes, infrastructure or interactions, or simply because they are tired or lack adequate training.

Defects:

By any types of error, it leads to defect. Defects can be found in documentation, such as a requirements specification or a test script, in source code, or in a supporting artifact such as a build file.

Failure:

Failure is the observable outcome of the bug when the software is executed in an operational environment, the system may fail to do what it should do, or do something it shouldn’t, causing a failure. If the software can not perform as per business need, then it is considered as failure. In another way, if the end user is not satisfied or found issues in the software, that is the ultimate failure.

Bug:

If a defect is caught by executing the test object and accepted by developer, then this is called bug. To prove the developers that it is a bug, needs bug advocacy.

Errors and defects are not the only cause of failures. Failures can also be caused by environmental conditions, such as hardware or low quality elements.

7 Testing principal

1. Testing shows presence of defects, not their absence

Meaning: Testing can find bugs, but it cannot prove the system is 100% bug-free.

Example: Even after full QA of a payment system, a rare edge-case transaction failure may still appear in production due to an untested network condition.

2. Exhaustive testing is impossible

Meaning: You cannot test all inputs, combinations, and scenarios.

Example: A login form with multiple browsers, devices, password combinations, and network conditions makes it impossible to test every single possibility.

3. Early testing saves cost and time

Meaning: Defects found earlier are cheaper and easier to fix.

Example: A requirement mistake found during design review costs minutes to fix, but the same bug found after release may require major code rewrite and deployment rollback.

4. Defects cluster together (Pareto principle)

Meaning: Most bugs come from a small number of modules.

Example: In an e-commerce app, 70% of bugs come from payment and checkout modules, while static pages have very few issues.

5. Tests wear out (Pesticide paradox)

Meaning: Repeating the same tests will stop finding new bugs over time.

Example: Running the same regression suite every release may miss new edge cases in a newly added discount feature.

6. Testing is context dependent

Meaning: Testing approach changes based on the application type and risk.

Example: Banking software requires strict security and accuracy testing, while a gaming app focuses more on performance and user experience.

7. Absence-of-errors fallacy

Meaning: A bug-free system is still useless if it does not meet user needs.

Example: A travel app with zero bugs is still a failure if it does not show correct flight prices or lacks essential booking features users expect.